

How many fixed-point bits do the spiking probabilities need?

6 October 2026

We want to represent the random walker’s spiking probabilities as integer codes with an agreed scale, and measure how many bits give useful accuracy. The CPU prepares the probabilities. Neuron states and walker counts remain integers: a probability controls whether a whole spike occurs, not a fractional state update.

The main result: across nine specific cases, nearest rounding with 15 binary fractional bits gives a largest average error of 0.928% and a largest peak error of 2.025%. With 16 fractional bits, these fall to 0.341% and 0.853%. Our format includes probability exactly 1, so these choices use 16 and 17 stored bits per probability. All stored probabilities use the same precision in each experiment. Decimal rounding is included as a supporting comparison.

1. What the random walker calculates

Start with 300 walkers in direction 0 and 180 in direction 1. A walker is one unit of population. Each step, it stays, reverses direction or is lost. The third state records losses during the latest step. We use the population at the start of the step, so each walker moves at most once.

The tutorial uses scattering rate 8, absorption rate 2 and time step 0.005. Scattering affects reversals; absorption affects loss. A run to time 5 takes 1,000 steps. We follow the number of walkers remaining in each direction. Walker counts stay whole numbers. Only the precision of the probabilities is changed in these experiments.

2. Which probabilities are rounded?

The original walker tries destinations in order. Suppose their overall chances are $[0.2, 0.3, 0.5]$. First choose destination 1 with chance 0.2. If that fails, choose destination 2 with chance $0.3/0.8 = 0.375$. Otherwise take destination 3. The second choice is reached 80% of the time, giving destination 2 its intended overall chance: $0.8 \times 0.375 = 0.3$.

The chance used after earlier choices fail is called a conditional probability. We call it q . This is the spiking probability we represent and round, once before the walk begins. The last destination takes the probability left over.

3. Store a probability as an integer

Choose a scale S , multiply q by S , and round to an integer T . The code T represents probability T/S . Nearest rounding uses

$$T = \lfloor Sq + 0.5 \rfloor,$$

where $\lfloor \cdot \rfloor$ means round down. This rounds exact half ties upward. Truncation instead uses $T = \lfloor Sq \rfloor$.

For $q = 2/3$	Scale S	Code T	Stored probability
Four decimal places	10,000	6,667	0.6667
12 binary fractional bits	4,096	2,731	0.666748046875

An integer with an agreed scale is fixed point. Binary fixed point uses a power-of-two scale.

How the notebook runs

The notebook runs on the CPU. `transport_matrix` builds the original movement probabilities. `rounded_matrix` rounds the conditional choices and rebuilds the resulting movement probabilities. `expectation` then calculates population curves step by step, once with each version. It multiplies counts by movement chances: 100 walkers with a 0.3 chance of moving give an expected 30 moves. It does not draw random moves or simulate individual spikes. Finally, `errors` compares the curves. The remaining loops repeat this calculation for the listed inputs and precisions, then make the tables and plots. Fractional expected counts are valid averages; they are not fractional hardware states.

How the comparison works

4. Fractional precision, stored bits and output scaling

With d decimal places, $S = 10^d$ and the spacing between probabilities is 10^{-d} . With F binary fractional bits, $S = 2^F$ and the spacing is 2^{-F} . An extra decimal place divides the spacing by ten; an extra binary bit divides it by two.

Why does our binary code need $F + 1$ stored bits? Take $F = 3$, so $S = 8$. Three bits store 0 through 7, representing probabilities up to $7/8 = 0.875$. Probability exactly 1 needs code 8, binary 1000, taking four bits. Thus $B = F + 1$ includes the endpoint; it does not make the spacing finer. These unsigned bit counts are per probability, with an agreed scale. An interface handling certainty separately may count storage differently. Four decimal places need 14 bits for codes 0 through 10,000.

The probability scale does not change the walker count. A count of 120 still means 120 walkers. The tutorial converts it to flux with $120/60 = 2$. Do not divide the count by S . Scaling a probability does not require shifting the walker count. The separate flux divisor 60 is not a power of two.

5. What average and peak error mean

“Float” uses the original probabilities; the rounded version uses their stored approximations. We calculate both expected-population curves directly, without random trials, at every time step including time zero: 1,001 time points for the original tutorial.

Compare the counts for each direction at each time. If Float gives 20 walkers and rounding gives 22, the absolute difference is 2, or $2/20 \times 100 = 10\%$ error. Average error gives equal weight to every included direction-and-time percentage. Peak error is the largest of them. We do not select a few sample times.

Small counts can make percentages look large: 1 walker instead of 0.5 is a 100% error. We therefore only include comparisons where Float is at least 10 when starting with 480 walkers. The cutoff scales with the starting population: 1 for 48 walkers, 10 for 480, and 100 for 4,800.

The notebook also checks every time point, including the late times excluded above, using a second measure. It divides each difference by the starting population: 2 walkers out of 480 is $2/480 \times 100 = 0.417\%$. This keeps the denominator fixed as walkers disappear.

6. Change the model inputs

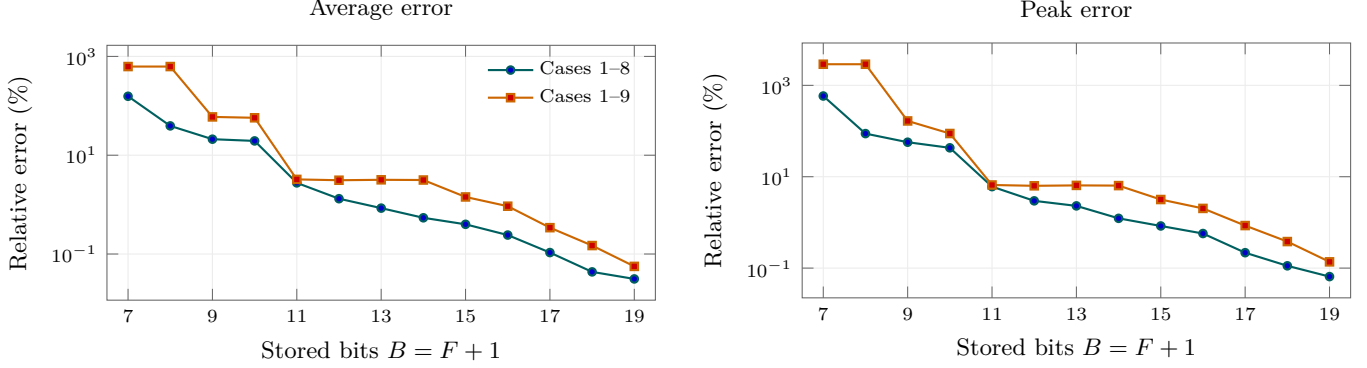
Each row is one input setting: 480 walkers, ending at time 5, with only the named input changed. Compare its rounded counts with its own Float counts. “Avg.” means mean percentage error across both directions and times with Float at least 10; “peak” means the largest such error. Both precisions use nearest rounding.

Case and change from the tutorial	$F=12$ avg. %	$F=12$ peak %	$F=16$ avg. %	$F=16$ peak %
(1) Original tutorial	0.602	1.289	0.069	0.145
(2) Scattering 4 instead of 8	0.849	2.300	0.025	0.060
(3) Scattering 16 instead of 8	0.220	0.549	0.045	0.093
(4) Absorption 1 instead of 2	0.287	0.709	0.107	0.217
(5) Absorption 4 instead of 2	0.212	0.572	0.020	0.029
(6) Time step 0.0025	0.572	1.292	0.081	0.183
(7) Time step 0.01	0.066	0.150	0.029	0.064
(8) Initial split [480, 0, 0]	0.550	1.230	0.064	0.139
(9) Time step 0.0005	3.178	6.460	0.341	0.853

“Eight nearby cases” means cases 1–8: the tutorial and seven variations. “All nine” adds case 9, the much smaller time step, requiring 10,000 steps. These are nine input settings, not nine random trials or a sweep of all possible inputs.

Compare every binary precision

At each precision, first calculate an average error and a peak error separately for each case, as in section 6. Then select the largest case-average and the largest case-peak. “Largest average” is therefore the worst of the case averages, not an average across cases. All results below use nearest rounding and the same Float-count cutoff of 10.



The teal curve selects the largest errors among cases 1–8; the orange curve includes case 9 too. Right means more bits; down means less error on a logarithmic scale. The table uses all nine cases and identifies the case producing each reported value.

F	Stored bits B	Largest avg. (%)	From case	Largest peak (%)	From case
0	1	717.107	8	4676.737	8
1	2	717.107	8	4676.737	8
2	3	717.107	8	4676.737	8
3	4	717.107	8	4676.737	8
4	5	717.107	8	4676.737	8
5	6	623.890	6	3172.673	2
6	7	623.317	9	2898.820	9
7	8	623.317	9	2898.820	9
8	9	59.607	9	166.240	9
9	10	57.222	9	88.304	9
10	11	3.240	9	6.594	9
11	12	3.116	9	6.326	9
12	13	3.178	9	6.460	9
13	14	3.147	9	6.393	9
14	15	1.429	9	3.169	9
15	16	0.928	9	2.025	9
16	17	0.341	9	0.853	9
17	18	0.148	9	0.381	9
18	19	0.056	9	0.137	9

For example, at $F = 12$, cases 1–8 give largest average/peak errors of 0.849%/2.300%, both from case 2. Including case 9 raises them to 3.178%/6.460%. At $F = 15$, the reported 0.928%/2.025% also both come from case 9. At $F = 5$, the two maxima come from different cases.

The notebook tests $F = 0$ through 18 and 0 through 6 decimal places with nearest rounding and truncation: $9 \times (19 + 7) \times 2 = 468$ comparisons. The tables round results to three decimals. Errors can exceed 100% when Float counts are small; adding a bit need not improve every error because different rounding errors can partly cancel.

Decimal rounding and example outputs

7. Compare decimal places

This supporting comparison uses nearest rounding and reports the largest average and peak among all nine cases. It does not require decimal arithmetic in hardware.

Decimal places d	Stored bits	Largest average (%)	Largest peak (%)
0	1	717.107	4676.737
1	4	717.107	4676.737
2	7	623.317	2898.820
3	10	3.596	7.762
4	14	0.631	1.443
5	17	0.366	0.801
6	20	0.008	0.018

Why can a coarser decimal grid do better?

Binary $F = 15$ has spacing $1/32768 \approx 0.00003052$; four decimal places have spacing 0.0001. Binary has finer spacing and a smaller maximum possible rounding error for one probability. But the grids put their values in different places. A particular probability can be closer to a decimal value.

For example, one conditional chance in the smallest-step case is 0.9990030035. Binary stores 0.9989929199; decimal stores 0.9990000000, which is closer. All conditional choices affect movement and loss, so their errors can reinforce or partly cancel over repeated steps.

Case	$F=15$ avg. %	$F=15$ peak %	$d=4$ avg. %	$d=4$ peak %
Tutorial	0.057	0.131	0.088	0.190
Half time step	0.242	0.571	0.581	1.443
Ten-times smaller step	0.928	2.025	0.631	1.282

Binary wins on both metrics in eight of nine cases. The smallest step sets both binary maxima. Decimal's largest average comes from that case, but its largest peak comes from the half-time-step case. Smaller worst-case errors in this test set do not make decimal generally better.

The rounding rule also matters: at $F = 12$, truncation raises the largest peak from 6.460% to 38.027%. All truncation results are in the notebook.

8. Look at the resulting walker counts

This example uses the original tutorial at time 2, after 400 steps. The entries are expected counts; total live walkers is the sum of the two directions.

Setting	Direction 0	Direction 1	Total live walkers
Float	5.246	5.246	10.492
$F=0$	300.000	180.000	480.000
$F=8$	8.371	8.128	16.499
$F=12$	5.173	5.165	10.338
$F=16$	5.255	5.255	10.510

Float leaves about 10.492 walkers in total. At $F = 0$, rounding removes the chance of loss, so all 480 remain. More precision brings the counts closer to Float. These are averages, so fractions are valid. Divide each direction's count by 60 for flux. Counts below 10 are shown here, although they are excluded from the relative-error summaries.

Run duration, population and the final choice

9. Error can build up over more steps

Keep the original probabilities, 480 walkers and time step 0.005. Change only the stopping time. The table uses nearest rounding; the notebook also includes $F = 8, 10$ and 14 .

Stopping time	$F=12$ avg. %	$F=12$ peak %	$F=16$ avg. %	$F=16$ peak %
0.25	0.085	0.227	0.009	0.023
0.5	0.174	0.424	0.020	0.045
1.0	0.357	0.799	0.041	0.088
2.0	0.602	1.289	0.069	0.145
5.0	0.602	1.289	0.069	0.145

Error can build up over more steps. The values at times 2 and 5 match because later Float counts are below 10 and are left out of this percentage. This does not mean later errors stop growing. The notebook's error divided by the initial population includes those later times.

10. More walkers give the same percentage error

Use 48, 480 or 4,800 walkers with the same 5 : 3 split between directions and stop at time 5. Ten times as many walkers gives ten times the count difference and ten times the Float count. Dividing one by the other gives the same percentage. Multiply the cutoff by ten too, so the comparisons cover the same times.

Initial N	Cutoff	$F=12$ avg. %	$F=12$ peak %	$F=16$ avg. %	$F=16$ peak %
48	1	0.602	1.289	0.069	0.145
480	10	0.602	1.289	0.069	0.145
4800	100	0.602	1.289	0.069	0.145

Adding walkers alone does not require more probability bits in this experiment. The count still needs enough storage for its largest value. If the three populations represent the same physical flux, divide their counts by 6, 60 and 600, respectively. This conversion is separate from the probability scale.

Final result: the binary fixed-point choice

For all nine cases, $F = 15$ is a useful starting point: store $T = \lfloor 32768q + 0.5 \rfloor$ in 16 bits. The largest average and peak errors are 0.928% and 2.025%. Using $F = 16$ costs one more stored bit and reduces them to 0.341% and 0.853%. $F = 14$ uses 15 stored bits and gives 1.429% and 3.169%. There is no single minimum without choosing how much error is acceptable; these measured choices make the tradeoff explicit.

The smaller $F = 12$ choice applies only to the eight nearby cases. All conclusions concern expected populations for the listed cases and the stated cutoff. Individual random runs can differ. Hardware implementation, integer-state sizing and error in the underlying physics approximation are separate questions.

Source and reproduction

The model comes from the original Sango/Brian2 random-walker tutorial. The companion notebook, `rw_bit_allocation_tutorial.ipynb`, contains every experiment and its saved output. All 14 code cells were run from a fresh kernel on 6 October 2026: 468 precision comparisons, 25 duration comparisons, 6 population comparisons and 5 output examples. The report uses those results.